INTRODUCCIÓN A JAVA

Programación



PARTE II

INDICE

ESTRUCTURAS ALTERNATIVAS **3**

ESTRUCTURA ALTERNATIVA SIMPLE (IF) 3
ESTRUCTURA ALTERNATIVA DOBLE (IF-ELSE) 4
ESTRUCTURA ALTERNATIVA MÚLTIPLE (SWITCH) 6

ESTRUCTURAS REPETITIVAS **8**

BUCLE FOR 9
BUCLE WHILE 11
BUCLE DO-WHILE 12

SENTENCIAS DE SALTO **13**

SENTENCIA RETURN 13
SENTENCIA BREAK 13
SENTENCIA CONTINUE 14

ESTRUCTURAS ALTERNATIVAS

Como vimos en el tema anterior de Pseudocódigo, las estructuras alternativas son construcciones que permiten alterar el flujo secuencial de un programa de forma que en función de una condición o el valor de una expresión, el mismo pueda ser desviado en una u otra alternativa de código.

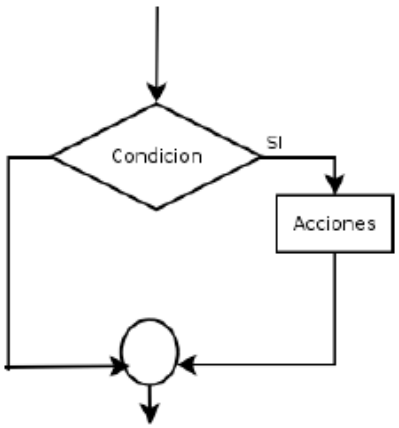
Las estructuras alternativas disponibles en Java son:

- Alternativa **Simple** (if)
- Alternativa **Doble** (if-else)
- Alternativa **Múltiple** (switch)

ESTRUCTURA ALTERNATIVA SIMPLE (IF)

La alternativa simple se codifica de la siguiente forma:

CÓDIGO	DIAGRAMA DE FLUJO

if (condición)
{
// Acciones;
}
 	
El bloque de Acciones se ejecuta si la condición se evalúa a true (es verdadera).
 	
if (cont == 0)
{
System.out.println("cont es 0");
// más instrucciones...
}
 	
Si dentro del if solo hay una instrucción, no es necesario poner las llaves.
if (cont == 0)
System.out.println("cont es 0");
	

ESTRUCTURA ALTERNATIVA DOBLE (IF-ELSE)

La alternativa doble permite indicar qué código ejecutar si la condición es falsa.

<table>
<tr><th>CÓDIGO</th><th>DIAGRAMA DE FLUJO</th></tr>
<tr><td>

```
if (condición)
{
        // AccionesSI;
}
else
{
        // AccionesNO;
}
```

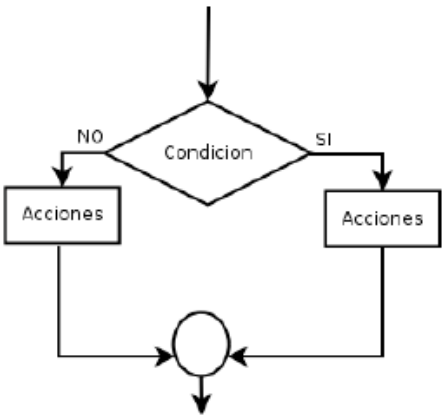
El bloque AccionesSI se ejecuta si la condición se evalúa a true (verdadera). En caso contrario, se ejecuta el bloque de AccionesNO.

```
if (cont == 0)
{
        System.out.println("cont es 0");
        // más instrucciones...
}
else
{
        System.out.println("cont no es 0");
        // más instrucciones...
}
```

Si dentro del if o el else solo hay una instrucción, no es necesario poner las llaves.

```
if (cont == 0)
        System.out.println("cont es 0");
else
        System.out.println("cont no es 0");
```

</td><td>



Recordad que el operador relacional para comprobar si son iguales es ==, no un solo = que corresponde con el operador de asignación. Este error no lo detecta el compilador y es difícil de averiguar.

</td></tr>
</table>

En muchas ocasiones, se anidan estructuras alternativas if-else, de forma que se pregunte por una condición si anteriormente no se ha cumplido otra sucesivamente.

Por ejemplo: *supongamos que realizamos un programa que muestra la nota de un alumno en la forma (insuficiente, suficiente, bien, not able o sobresaliente) en función de su nota numérica.*

Podría codificarse de la siguiente forma:

```
1
2  import java.util.Scanner;
3
4  public class Nota {
5
6      public static void main(String[] args) {
7          Scanner entrada = new Scanner(System.in);
8          int nota;
9          //Suponemos que el usuario introduce el número correctamente.
10         //No hacemos comprobación
11         System.out.println("Dame un número entre 0 y 10");
12
13         nota = entrada.nextInt();
14
15         if (nota < 5) {
16
17             System.out.println("Insuficiente");
18
19         } else if (nota < 6) {
20
21             System.out.println("Suficiente");
22
23         } else if (nota < 7) {
24
25             System.out.println("Bien");
26
27         } else if (nota < 9) {
28
29             System.out.println("Notable");
30
31         } else {
32
33             System.out.println("Sobresaliente");
34         }
35
36     }
37
38 }
```

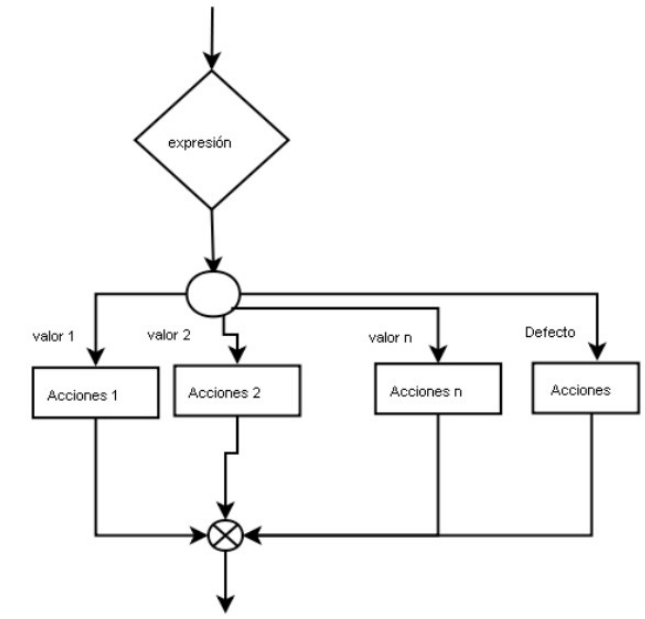
Siendo la salida:

```
run:
Dame un número entre 0 y 10
8
Notable
BUILD SUCCESSFUL (total time: 11 seconds)
```

Es muy recomendable usar la tecla tabulador en las instrucciones de cada bloque. Como se puede ver en el ejemplo, cada else está alineado con su if asociado, de esta forma es más fácil leer el código.

ESTRUCTURA ALTERNATIVA MÚLTIPLE (SWITCH)

La alternativa múltiple se codifica de la siguiente forma:

CÓDIGO	DIAGRAMA DE FLUJO
switch (expresión) *{* *case valor1:* *// Acciones1;* *break;* *case valor2:* *// Acciones2;* *break;* *case valorN:* *// AccionesN;* *break;* *default:* *// Acciones por defecto;* *}*	

Es muy importante entender que en el switch se evalúa una expresión (un valor concreto como 0, 5, 1…) no una condición (verdadera o falsa) como en el if y el ifelse.

El programa comprueba el valor de la expresión y saltará al 'case' que corresponda con dicho valor (valor1 o valor2 o …) ejecutando el código de dicho 'case' (Acciones1 o Acciones2 o …). Si no coincide ningún valor, saltará al 'default' y ejecutará las acciones por defecto.

Es importante añadir la sentencia break; al final de cada 'case', ya que de lo contrario el programa seguirá ejecutando el código de las demás acciones y normalmente no querremos que haga eso (aunque Java permite hacerlo, es confuso y por ello está desaconsejado).

Un ejemplo sería el siguiente:

```java
import java.util.Scanner;

public class Alternativa_Multiple {

    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int dia;

        System.out.println("Dame un número entre 1 y 7:");

        dia = entrada.nextInt();

        switch (dia) {
            case 1:
                System.out.println("Lunes");
                break;
                case 2:
                System.out.println("Martes");
                break;
                case 3:
                System.out.println("Miércoles");
                break;
                case 4:
                System.out.println("Jueves");
                break;
                case 5:
                System.out.println("Viernes");
                break;
                case 6:
                System.out.println("Sábado");
                break;
                case 7:
                System.out.println("Domingo");
                break;
            default:
                System.out.println("Error el número debe estar entre 0 y 7");
        }

    }

}
```

Y la salida:

```
run:
Dame un número entre 1 y 7:
4
Jueves
BUILD SUCCESSFUL (total time: 2 seconds)
```

ESTRUCTURAS REPETITIVAS

Los bucles son estructuras de repetición, bloques de instrucciones que se repiten un número de veces mientras se cumpla una condición o hasta que se cumpla una condición.

Un bloque de instrucciones se encontrará encerrado mediante llaves {.....} si existe más de una instrucción al igual que suceden las estructuras alternativas (if… else… etc).

Existen tres construcciones para estas estructuras de repetición:

- Bucle for
- Bucle while
- Bucle do-while

Todo problema que requiera repetición puede hacerse con cualquiera de los tres, pero según el caso suele ser más sencillo o intuitivo utiliza uno u otro.

Como regla general es recomendable:

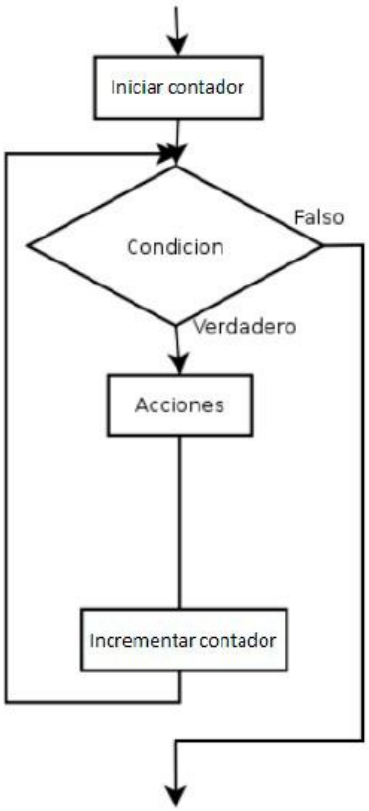
- Utilizar el bucle for cuando se conozca de antemano el número exacto de veces que ha de repetirse el bloque de instrucciones.
- Utilizar el bucle while cuando no sabemos el número de veces que ha de repetirse el bloque y es posible que no deba ejecutarse ninguna vez.
- Utilizar el bucle do-while cuando no sabemos el número de veces que ha de repetirse el bloque y deberá ejecutarse al menos una vez.

Estas reglas son generales y algunos programadores se sienten más cómodos utilizando principalmente una de ellas. Con mayor o menor esfuerzo, puede utilizarse cualquiera de las tres indistintamente.

BUCLE FOR

El bucle for se codifica de la siguiente forma:

CÓDIGO	DIAGRAMA DE FLUJO

for (inicialización ; condición ; incremento)
{
bloque acciones;
}
 	
La cláusula **inicialización** es una instrucción que se ejecuta una sola vez al inicio del bucle, normalmente para inicializar un contador.
Por ejemplo int i = 1;
 	
La cláusula **condición** es una expresión lógica que se evalúa al inicio de cada iteración del bucle. En el momento en que dicha expresión se evalúe a false se dejará de ejecutar el bucle y el control del programa pasará a la siguiente instrucción (a continuación del bucle for). Se utiliza para indicar la condición en la que quieres que el bucle continúe.
Por ejemplo i <= 10;
 	
La cláusula **incremento** es una instrucción que se ejecuta al final de cada iteración del bucle (después del bloque de instrucciones). Generalmente se utiliza para incrementar o decrementar el contador.
Por ejemplo i++; (incrementar i en 1).
	

Ejemplo 1: *Bucle que muestra por pantalla los números naturales del 1 al 10:*

```
for (int i = 1; i <= 10 ; i++) {
        System.out.println(i);
}
```

En la inicialización utilizamos int i=1 para crear la variable i con un valor inicial de 1.

La condición i<=10 indica que el bucle debe repetirse mientras i sea menor o igual a 0.

La actualización i++ indica que, al final de cada iteración, i debe incrementarse en 1.

Ejemplo 2: *Programa que muestra los números naturales (1,2,3,4,5,6,...) hasta un número introducido por teclado.*

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int max;
        System.out.print("Introduce el número máximo: ");
        max = sc.nextInt();
        for (int i = 1; i <= max; i++) {
            System.out.println("Número: " + i);
        }
    }
}
```

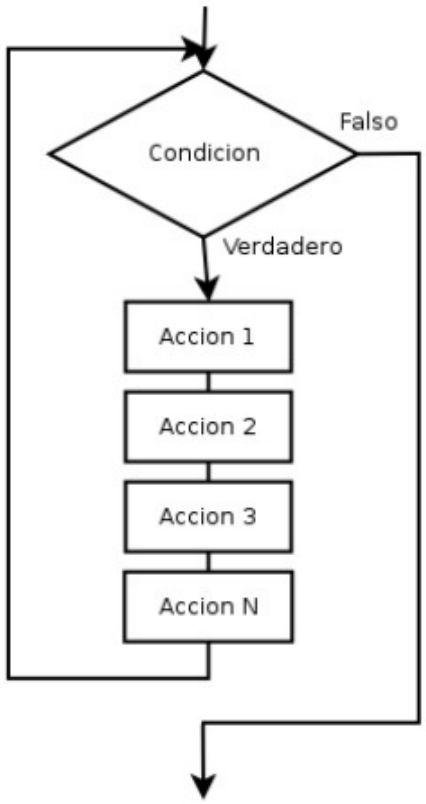
Siendo la salida:

```
run:
Introduce el número máximo: 5
Número: 1
Número: 2
Número: 3
Número: 4
Número: 5
BUILD SUCCESSFUL (total time: 6 seconds)
```

BUCLE WHILE

El bucle while se codifica de la siguiente forma:

CÓDIGO	DIAGRAMA DE FLUJO

while (condición) {
bloque acciones;
}
 	
El bloque de instrucciones **se ejecuta mientras se cumple una condición** (mientras condición se
evalúe a true).
 	
La condición se **comprueba ANTES de empezar** a ejecutar por primera vez el bucle, por lo que si se evalúa a false en la primera iteración, entonces el bloque de acciones no se ejecutará ninguna vez.
	

El mismo ejemplo 2 anterior hecho con un bucle while sería:

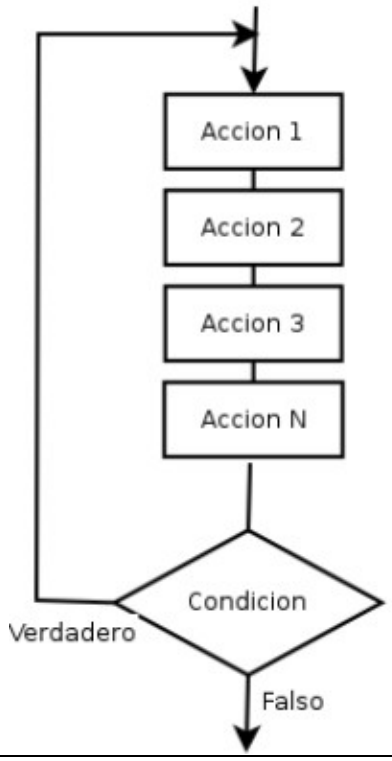
```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int max, cont;
        System.out.print("Introduce el número máximo: ");
        max = sc.nextInt();
        cont = 1;
        while (cont <= max) {
            System.out.println("Número: " + cont);
            cont++;
        }
    }
}
```

Y la salida:

```
run:
Introduce el número máximo: 5
Número: 1
Número: 2
Número: 3
Número: 4
Número: 5
BUILD SUCCESSFUL (total time: 6 seconds)
```

BUCLE DO-WHILE

El bucle do-while se codifica de la siguiente forma:

CÓDIGO	DIAGRAMA DE FLUJO
do { bloque acciones; }while (condición); En este tipo de bucle, **el bloque de instrucciones se ejecuta siempre al menos una vez**, y ese bloque de instrucciones se ejecutará mientras condición se evalúe a true. Por ello en el bloque de instrucciones deberá existir alguna instrucción que, en algún momento, haga que condición se evalúe a false. ¡Si no el bucle no acabaría nunca!	

El mismo ejemplo 2 anterior hecho con un bucle do-while sería:

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int max, cont;
        System.out.print("Introduce el número máximo: ");
        max = sc.nextInt();
        cont = 1;

        do {
            System.out.println("Número: " + cont);
            cont++;
        } while (cont <= max);

    }
}
```

Y la salida:

```
run:
Introduce el número máximo: 5
Número: 1
Número: 2
Número: 3
Número: 4
Número: 5
BUILD SUCCESSFUL (total time: 6 seconds)
```

SENTENCIAS DE SALTO

En este apartado se tratan brevemente las instrucciones que Java proporciona para saltar de un punto del código a otro. Son las instrucciones break, continue y return.

SENTENCIA RETURN

La sentencia return permite devolver el resultado obtenido al ejecutarse un método.

También puede situarse al final del programa principal, puesto que el main es también un método.

En el código de un método pueden aparecer varias instrucciones return. Sin embargo, en cada ejecución del método, sólo una de las instrucciones return se ejecutará, dado que la misma siempre supone la conclusión del método.

SENTENCIA BREAK

La sentencia break ya se ha visto en su uso para interrumpir la ejecución de una sentencia switch. En esa situación, la ejecución del break supone saltar a la primera instrucción que esté justo después del final de la estructura switch.

Además, break puede usarse dentro de los bucles para salir de los mismos sin verificar la condición de finalización del bucle.

Ejemplo:

```
int dia = 3;
for (int i = 1; i <= 7; i++) {
        if (dia == i){
                System.out.println("Hoy es el " + i + "º dia de la semana.");
                break;
        }
        System.out.println("Dia " + i);
}
System.out.println("Seguimos...");
```

SENTENCIA CONTINUE

La sentencia continue sirve, dentro de bucles, para salir anticipadamente de la iteración del bucle y continuar con la ejecución de la siguiente iteración del mismo bucle.

Ejemplo:

```
int dia = 3;
for (int i = 1; i <= 7; i++) {
        if (dia == i){
                continue;
        }
        System.out.println("Dia " + i);
}
System.out.println("Seguimos...");
```